

TCP Congestion Control

Introduction/Flow Control vs Congestion Control:

Using flow control in TCP, the size of the send window is controlled by the receiver using the value of `rwnd`, which is advertised in each segment traveling in the opposite direction. The use of this strategy guarantees that the receive window is never overflowed with the received bytes (no end congestion). This, however, does not mean that the intermediate buffers, buffers in the routers, do not become congested. A router may receive data from more than one sender. No matter how large the buffers of a router may be, it may be overwhelmed with data, which results in dropping some segments sent by a specific TCP sender. In other words, there is no congestion at the other end, but there may be congestion in the middle. Controlling or minimizing the congestion in the middle is known as congestion control.

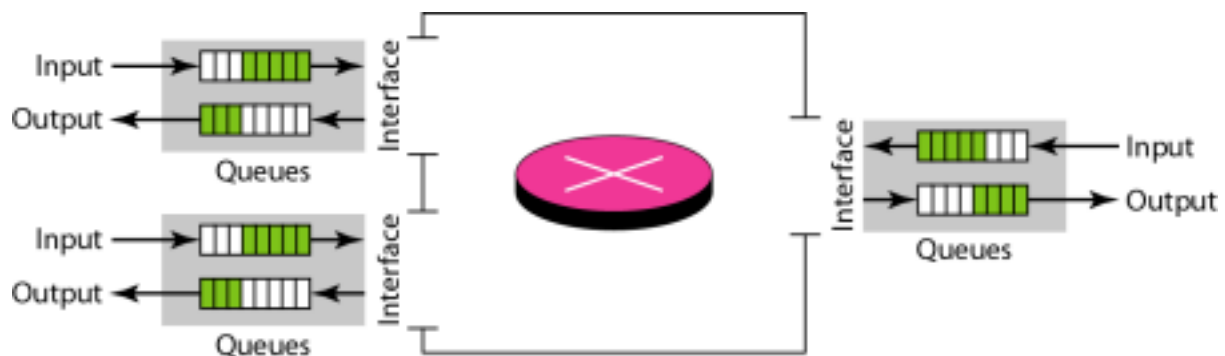
Necessity of Congestion Control

TCP needs to worry about congestion in the middle because many segments lost may seriously affect the error control. More segment loss means resending the same segments again, resulting in worsening the congestion, and finally, the collapse of the communication. TCP is an end-to-end protocol that uses the service of IP. The congestion in the router is in the IP territory and should be taken care of by IP. IP is a simple protocol with no congestion control. TCP, itself, needs to be responsible for this problem.

TCP cannot ignore the congestion in the network; it cannot aggressively send segments to the network. The result of such aggressiveness would hurt the TCP itself, as we mentioned before. TCP cannot be very conservative, either, sending a small number of segments in each time interval, because this means not utilizing the available bandwidth of the network. TCP needs to define policies that accelerate the data transmission when there is no congestion and decelerate the transmission when congestion is detected.

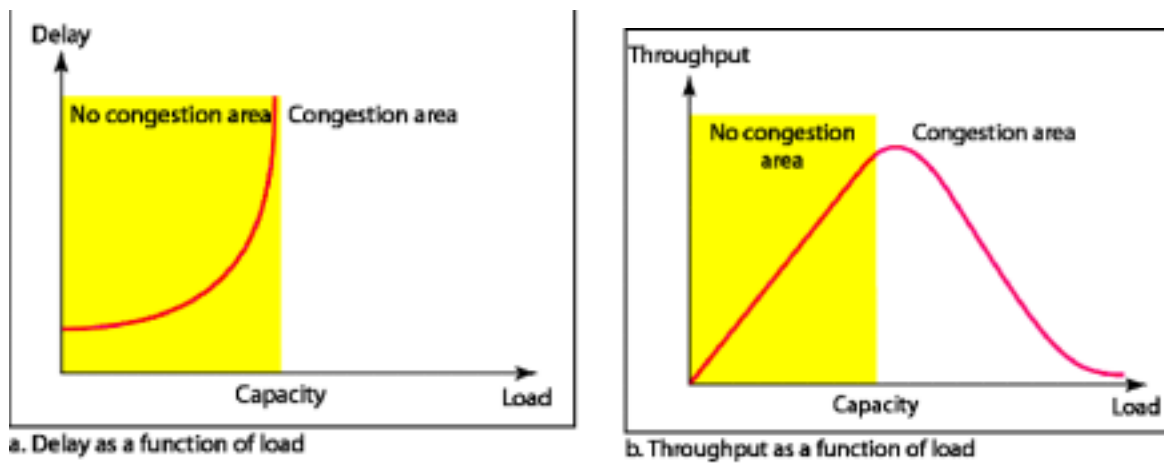
When Congestion Occurs

Congestion occurs when the load on the network is higher than the capacity of the network. In other words, each network has a capacity or a threshold of how many packets it can process or handle in a given amount of time. If the number of packets sent to that network tops the threshold, congestion occurs. This situation can be described using the input and output queues of routers.



Every interface of a router has one input queue for the incoming packets and one output queue for the outgoing packets. If the number of packets coming to the router exceeds the processing speed of the router, the input queue will be longer. On the other hand, if the processing speed is higher than the departure rate, the output queue will be longer. To understand how the queues get longer, let's look at an example. Imagine you went to a restaurant which can serve 100 customers at a time. However, you went there with all of your friends and other people, making the count 200 people. Since the restaurant can serve only 100 people at a time, some of you need to wait in the lounge to be served. Now, if other people also come at that time, the number of people waiting in the lounge will become higher and higher. Thus, it contributes to the queue being longer. Just like the restaurant, all intermediary devices have a capacity which can't be exceeded.

Congestion greatly affects the performance of a network. If we use delay and throughput as the performance metric of a network, we would get such graphs. In the left graph, it is visible that as the load approaches the capacity of the network, the delay becomes higher and the rate is exponential. Once it reaches the capacity, the line



becomes so steep that the delay becomes close to infinite, resulting in a drop of packets. On the right side, we can see that as the load approaches the capacity, the throughput increases linearly. Once it exceeds the capacity, the throughput starts decreasing sharply. This can be understood if we look at the results of congestion. Due to congestion, there will be more retransmissions and more delay in the router queues, eventually dropping the packets. If the packet drop increases, the number of successful data packets sent to the receiver will decrease, resulting in lower throughput. So, although users send more data, less of it actually gets delivered successfully. That's why throughput doesn't just flatten — it can actually drop steeply after the congestion point.

Congestion Control Categories:

Considering the serious effects of congestion control, it needs to be minimized. The only way to minimize congestion is by keeping the load below the capacity. We can achieve that in two ways, known as,

1. Open Loop Congestion Control
2. Closed Loop Congestion Control

More about them in the next section.

Open Loop Congestion Control:

Open loop congestion control is like preventing a disease before it occurs. These strategies work to make sure that congestion doesn't occur. Open loop congestion control policies are listed below.

- **Retransmission Policy:** When the sender assumes that a packet has been lost, it

retransmits the packet. Now, considering the congestion that might happen, it is going to worsen the situation. It's like sending a car on a crowded road to check if there is any traffic jam or not. The car itself will contribute to congestion. So, the retransmission policy should be set in a way that doesn't contribute to congestion.

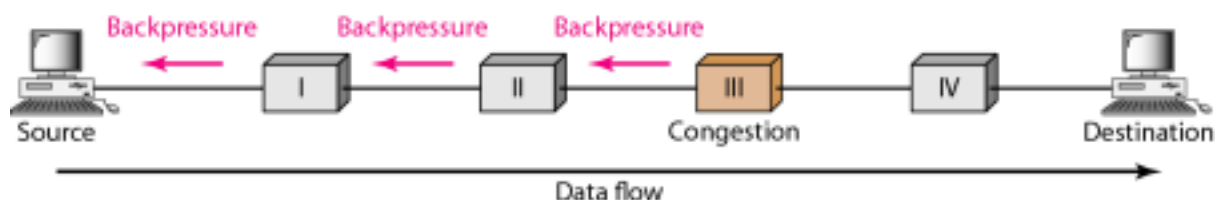
- **Window Policy:** We are already familiar with two types of sliding window protocols in TCP, known as selective repeat and go-back-n. Between these two, go-back-n worsens the congestion, as it forces the sender to resend all the segments starting from the missing segment. Considering this, the selective repeat is a much better window policy to minimize congestion.
- **Acknowledgment Policy:** Acknowledgment packets also contribute to the congestion. Also, later, we will see that the sender increases its sending rate based on the number of acknowledgments received. So, not acknowledging every packet slows down the sender and helps prevent congestion.
- **Discarding Policy:** Routers may choose to drop some packets to prevent congestion. Like, if an audio is being sent, the router may somehow detect some data that is less sensitive and drop that data. A good discarding policy by routers may prevent congestion and, at the same time, may not harm the integrity of the transmission.
- **Admission Policy:** To understand this policy, we need to dive a little into packet switching. There are two types of networks to switch packets, one which requires a fixed connection/path between the sender and the receiver, known as virtual circuit, and the other one doesn't require any fixed path, known as datagram network. Now our focus is on the virtual circuit network, which creates a connection, like setting up a call before sending the packets, and it will follow the same path to send all of the packets. Now, a router may sense congestion if it allows any new connection establishment. By using an admission policy, it may not allow new virtual circuit connections as long as it considers that a connection may introduce congestion to the network.

Closed Loop Congestion Control:

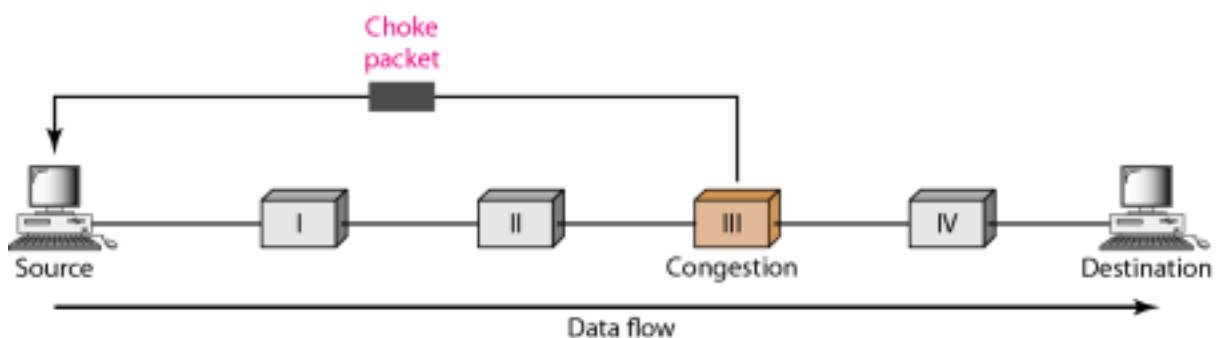
Closed loop congestion control is more like curing a disease once it has occurred. These strategies work to remove the congestion. If we consider a situation where a road is full of traffic, people will try to avoid that road. So, less number of people will go through that

road, and it will eventually lower the load on that road, making the traffic become normal and less crowded. The same approach is followed here, too. The sender will somehow get to know that congestion occurred, and it will lower the rate of sending data, resulting in congestion to disappear. Closed loop congestion control policies are listed below.

- **Backpressure:** The congested node/router (Router III in the picture) stops receiving data from the upstream or previous node. It is like a traffic police in a road junction is calling the traffic police of the previous junction to send fewer cars toward the congested road. Now, since the upstream/previous node (Router II in the picture) can't transfer the data, its output queue becomes longer, resulting in congestion in the upstream node (Router II). Then it (Router II) will again reject data from its upstream node (Router I). Thus, the backpressure slowly arrives at the sender, and the sender slows the rate of transmission. This can be used only in virtual circuit network, as each node has the information of its previous node.



- **Choke Packet:** Here, the congested node directly sends a specialized packet named “Choke packet” directly to the source. The intermediary nodes don't get any warning to lower the transmission rate. An example can be an ICMP source quench message, where the router sends a control packet to the source to slow down its sending rate.



- **Implicit Signaling:** No communication between the congested node or nodes happens with the source, yet the source can implicitly get a signal that it needs to slow down the transmission rate. Suppose you send 2 parcels per day to your friend, and he calls you back once he gets the parcels. It takes on average 2 hours to reach the parcels to your friend. Now, if you don't receive a call from your friend even after more than 2 hours, you will get a hint that your parcels haven't reached the destination. In this example, you are assuming that the parcels got dropped even if you didn't get any signal from the traffic or your friend. This is implicit signaling. When the sender receives a delayed acknowledgment or no acknowledgment for the sent packets, it

senses congestion and slows its transmission.

- **Explicit Signaling:** In this case, the congested node directly informs the source. This also happens in “Choke Packet” too. Then how is it different from Choke Packet? Remember, in case of choke packet, a specialized packet was created just to convey that congestion happened, but here the signal gets included in the data packet itself. Explicit signaling can be

- Backward Signaling - The Source gets warned, so the source slows down data transmission
- Forward Signaling - The Receiver gets warned, and the receiver slows down acknowledgment transmission.

Congestion Control in TCP

The sender's window size in TCP is not only determined by the receiver but also by congestion in the network. We are already familiar with `rwnd` set by the receiver, which helps in flow control. To control the number of segments to transmit, TCP uses another variable called a congestion window, `cwnd`, whose size is controlled by the congestion situation in the network. The `cwnd` variable and the `rwnd` variable together define the size of the send window in TCP. The first is related to the congestion in the middle (network); the second is related to the congestion at the end. The actual size of the window is the minimum of these two.

Actual window size = minimum (`rwnd`, `cwnd`)

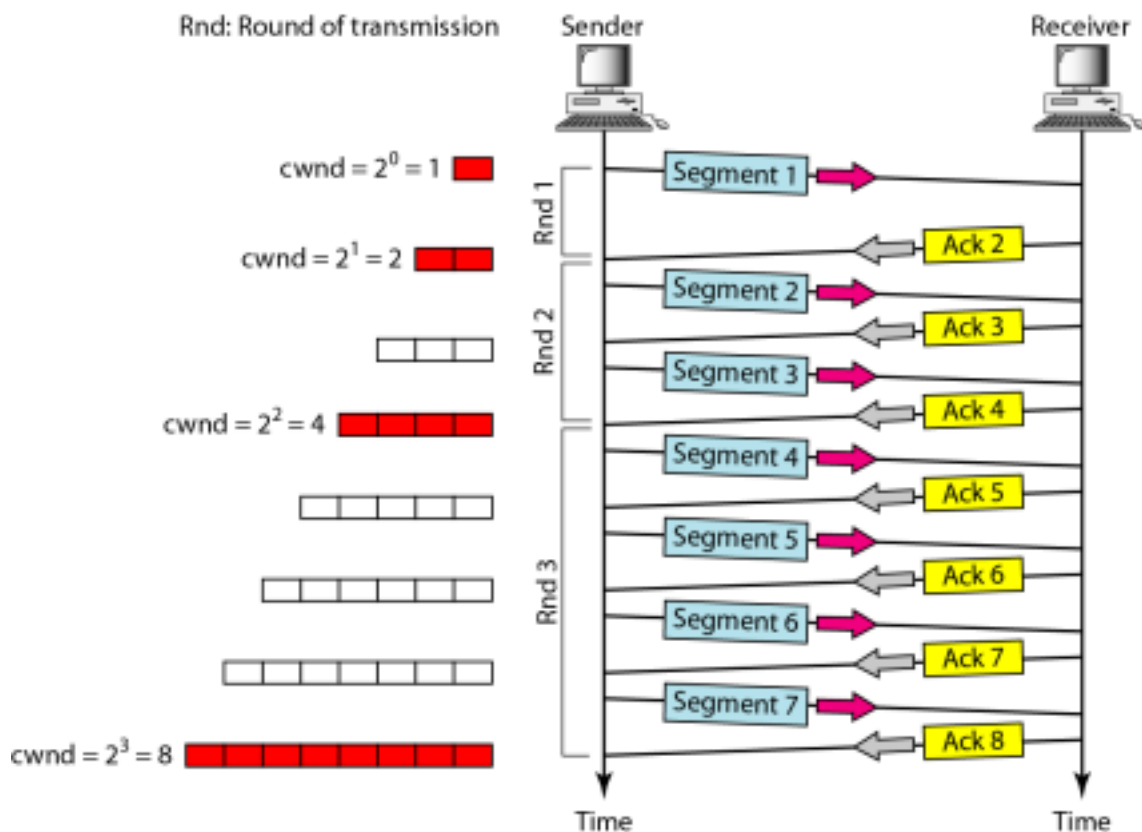
TCP does congestion control in three phases. These are

1. Slow Start
2. Congestion Avoidance
3. Congestion Detection

Slow Start: Exponential Increase

The slow-start algorithm is based on the idea that the size of the congestion window (`cwnd`) starts with one maximum segment size (MSS), but it increases one MSS each time an acknowledgment arrives. The MSS is a value negotiated during the connection establishment, using an option of the same name. The name of this algorithm is misleading;

the algorithm starts slowly, but grows exponentially. We assume that rwnd is much larger than cwnd , so that the sender window size always equals cwnd . We also assume that each segment is of the same size and carries MSS bytes. For simplicity, we also ignore the delayed-ACK policy and assume that each segment is acknowledged individually.



The sender starts with $\text{cwnd} = 1$. This means that the sender can send only one segment. After the first ACK arrives, the acknowledged segment is purged from the window, which means there is now one empty segment slot in the window. The size of the congestion window is also increased by 1 because the arrival of the acknowledgment is a good sign that there is no congestion in the network. The size of the window is now 2. After sending two segments and receiving two individual acknowledgments for them, the size of the congestion window now becomes 4, and so on. In other words, the size of the congestion window in this algorithm is a function of the number of ACKs arrived and can be determined as follows.

If an ACK arrives, $\text{cwnd} = \text{cwnd} + 1$.

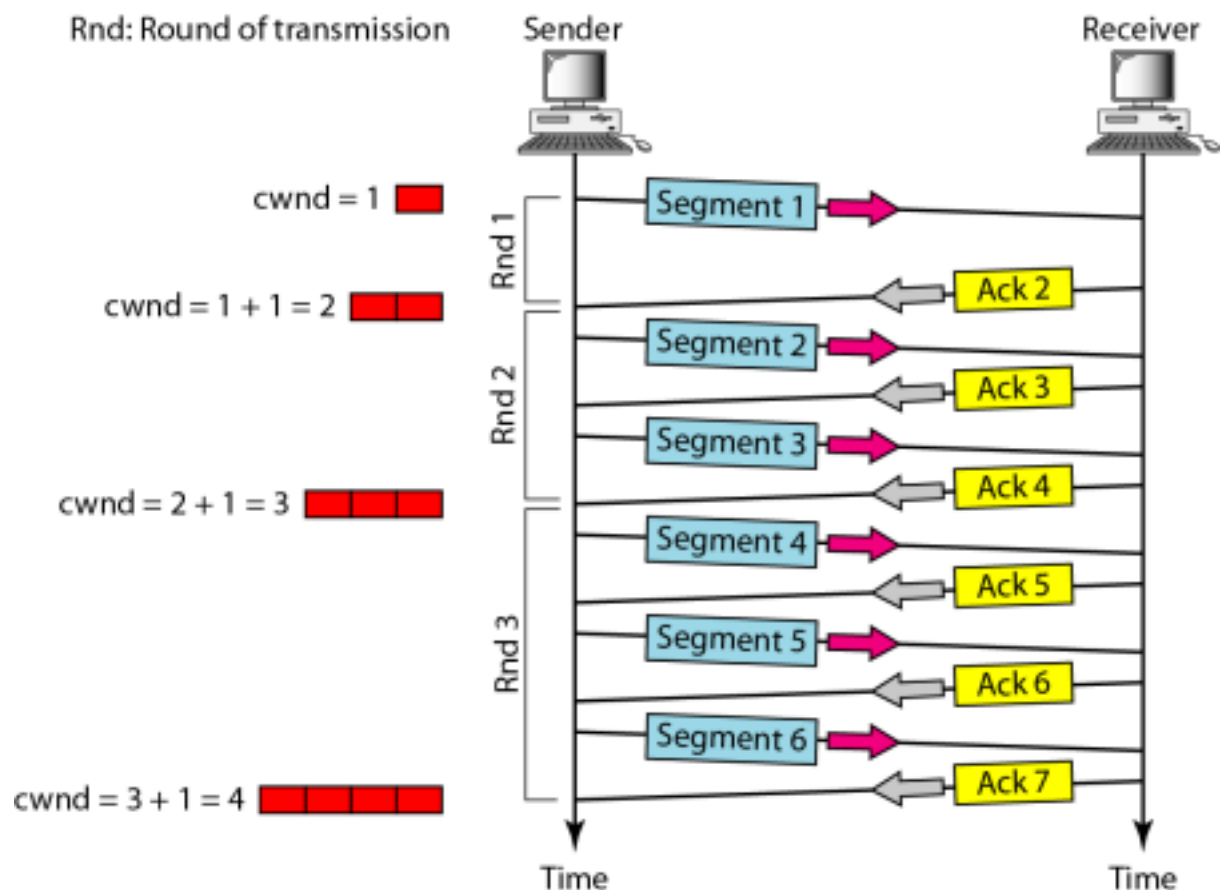
If we look at the size of the cwnd in terms of round-trip times (RTTs), we find that the growth rate is exponential in terms of each round trip time, which is a very aggressive approach:

Start	→	$cwnd = 1 \rightarrow 2^0$
After 1 RTT	→	$cwnd = cwnd + 1 = 1 + 1 = 2 \rightarrow 2^1$
After 2 RTT	→	$cwnd = cwnd + 2 = 2 + 2 = 4 \rightarrow 2^2$
After 3 RTT	→	$cwnd = cwnd + 4 = 4 + 4 = 8 \rightarrow 2^3$

A slow start cannot continue indefinitely. There must be a threshold to stop this phase. The sender keeps track of a variable named *ssthresh* (slow-start threshold). When the size of the window in bytes reaches this threshold, slow start stops, and the next phase starts. In the slow-start algorithm, the size of the congestion window increases exponentially until it reaches a threshold. We need, however, to mention that the slow-start strategy is slower in the case of delayed acknowledgments. Remember, for each ACK, the *cwnd* is increased by only 1. Hence, if two segments are acknowledged cumulatively, the size of the *cwnd* increases by only 1, not 2. The growth is still exponential, but it is not a power of 2. With one ACK for every two segments, it is a power of 1.5.

Congestion Avoidance: Additive Increase

If we continue with the slow-start algorithm, the size of the congestion window increases exponentially. To avoid congestion before it happens, we must slow down this exponential growth. TCP defines another algorithm called congestion avoidance, which increases the *cwnd* additively instead of exponentially. When the size of the congestion window reaches the slow-start threshold, the slow-start phase stops and the additive phase begins. In this algorithm, each time the whole “window” of segments is acknowledged, the size of the congestion window is increased by one. A window is the number of segments transmitted during RTT.



Consider the sender starts with $cwnd = 3$. This means that the sender can send only three segments. After three ACKs arrive, the acknowledged segments are purged from the window. Since the whole window of the segments has been acknowledged, the size of the congestion window is increased by 1. The size of window is now 4 (one added and 3 purged). After sending four segments and receiving four acknowledgments for them, the size of the congestion window now becomes 5, and so on. In other words, the size of the congestion window in this algorithm is also a function of the number of ACKs that have arrived and can be determined as follows:

If an ACK arrives, $cwnd = cwnd + (1/cwnd)$

The size of the window increases only $1/cwnd$ portion of MSS (in bytes). In other words, all segments in the previous window should be acknowledged to increase the window 1 MSS bytes. If we look at the size of the $cwnd$ in terms of round-trip times (RTTs), we find that the growth rate is linear in terms of each round-trip time, which is much more conservative

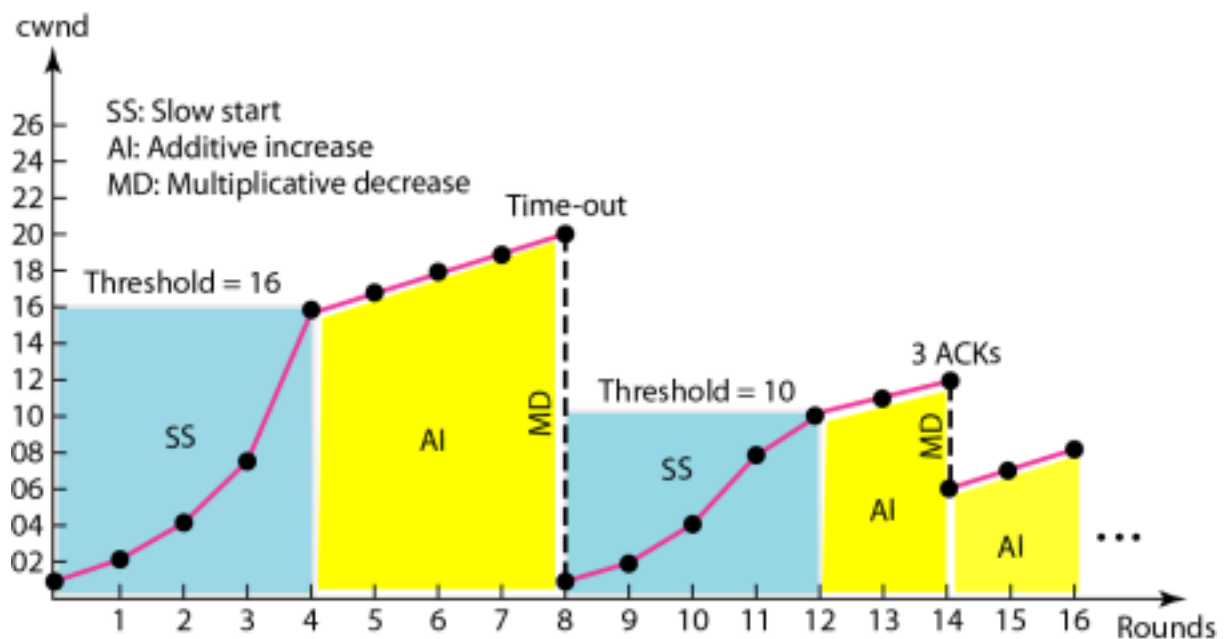
than the slow-start approach.

Start	→	$cwnd = i$
After 1 RTT	→	$cwnd = i + 1$
After 2 RTT	→	$cwnd = i + 2$
After 3 RTT	→	$cwnd = i + 3$

1. **cwnd=ssthresh:** TCP goes to Congestion Avoidance state where cwnd will be increased additively.
2. **Congestion is detected (occurrence of a time-out):** TCP immediately interrupts this aggressive growth and restarts a new slow start algorithm by limiting the threshold to half of the current cwnd and resetting the congestion window to 1. (ssthresh=cwnd/2, and cwnd = 1)
3. **Congestion is detected (arrival of three duplicate ACKs):** TCP moves to the Congestion Avoidance step, limiting the threshold to half of the current cwnd and resetting the congestion window to ssthresh. (ssthresh=cwnd/2, and cwnd = ssthresh)
4. **No more data to transmit:** Connection closed.

Once TCP reaches the Congestion Avoidance stage, there can be three scenarios like below.

1. **Congestion is detected (occurrence of a time-out):** TCP moves to a new slow start algorithm by limiting the threshold to half of the current cwnd and resetting the congestion window to 1. (ssthresh=cwnd/2, and cwnd = 1)
2. **Congestion is detected (arrival of three duplicate ACKs):** TCP stays in the Congestion Avoidance step, but cuts the threshold to half of the current cwnd and resets the congestion window to ssthresh. (ssthresh=cwnd/2, and cwnd = ssthresh)
3. **No more data to transmit:** Connection closed.



In the above example, TCP starts data transfer and sets the *ssthresh* variable to an ambitious value of 16 MSS. TCP begins at the slow-start (SS) state with the *cwnd* = 1. The congestion window grows exponentially and reaches the threshold in the 4th RTT. Then TCP moves to the congestion avoidance state and starts increasing *cwnd* additively. During the 8th RTT, Time out occurred when the *cwnd* was 20. So, TCP makes new *ssthresh* = $20/2 = 10$ and starts a new slow start, setting *cwnd* = 1. The window again grows exponentially and reaches the threshold at the 12th RTT. Then again, it goes to the congestion avoidance state and starts growing linearly. At the 14th RTT, when the *cwnd* was 12, congestion occurred due to 3 duplicate acks. TCP stays at the congestion avoidance state, but cuts the threshold and *cwnd* to half of the current *cwnd*. The updated *cwnd* becomes 6 and again starts increasing additively, and this goes on.

AIMD: Additive Increase Multiplicative Decrease

Most of the time, the congestion is detected and taken care of by observing the three duplicate ACKs. Even if there are some time-out events, TCP recovers from them by aggressive exponential growth. In other words, in a long TCP connection, if we ignore the slow-start states and short exponential growth during fast recovery, the TCP congestion window is $cwnd = cwnd + (1 / cwnd)$ when an ACK arrives (congestion avoidance), and $cwnd = cwnd / 2$ when congestion is detected, as though slow start state does not exist. The first is called additive increase; the second is called multiplicative decrease. This means that the congestion window size, after it passes the initial slow-start state, follows a saw tooth pattern called additive increase, multiplicative decrease (AIMD)

